CASE Template Module

A file-by-file guide for new developers

88 files - server, web console and tests

How to use this guide

Every file gets one card: what it is, what it does, and one idea worth remembering. Cards are grouped the way the code is grouped, so you can read a whole folder at once. Most files are small - read the card, then open the file next to it.

The shape of a request

1. The browser calls the API with an identity header.
2. Routes check the permission, then call a service.
3. The service validates with Zod, applies business rules, and talks to a repository.
4. Repositories are the only place SQL runs. Postgres keeps templates, runs and the audit chain.
5. Execution uses LangGraph: approvals and uploads pause the run until a person answers.
6. Every privileged action appends a hash-chained audit event.

Rule of thumb: HTTP handlers stay thin, services own decisions, repositories own SQL, and the domain folder owns vocabulary. If you are unsure where a change belongs, follow that chain.

Generated for the repository at Philosamphy/template-module-CASE

Contents

1. Start here
6 files

2. Server - boot and wiring
7 files

3. Server - domain
3 files

4. Server - database
9 files

5. Server - identity and permissions
2 files

6. Server - audit trail
2 files

7. Server - AI
4 files

8. Server - workflow execution
6 files

9. Server - repositories
3 files

10. Server - services
2 files

11. Server - routes
5 files

12. Server - tests
8 files

13. Web console
17 files

14. Appendix - the dashboard app
14 files

Start here

Documents and top-level configuration. Read these first if you want to run the module before understanding it.

01 **`README.md`**

The front door: what the module is, the stack, the folder map and how to run it.

- Explains the idea: operators build or describe a workflow, and nothing runs until a human approves it.
- Lists every environment variable you need before the first start.
- Keep this file honest - it is the first thing a reviewer opens.

02 **`DEMO.md`**

A short script for demonstrating the module to someone in a few minutes.

- Walks through create -> approve -> run -> respond with exact clicks.
- Useful as a smoke test: if the demo path breaks, the app is broken.

03 **`LAUNCH-AND-DEMO.md`**

The longer setup guide: database, model server, and both apps on one machine.

- Covers prerequisites (Node, PostgreSQL, Ollama model) and the commands to start each part.
- Troubleshooting section records the real problems hit during development.

04 **`.gitignore`**

Tells git what never to commit.

- Keeps out node_modules/, build output, logs and the .env file with real credentials.
- Also excludes generated documents and the multi-megabyte OCR language data.
- If a secret ever appears in git history, removing the file is not enough - rotate the secret.

05 **`package.json`**

The workspace root: two packages (server, web) and the npm scripts that drive them.

- npm workspaces let one install cover both packages and share one lockfile.
- Scripts are shortcuts: dev:server, dev:web, build, test, migrate.
- Read the scripts before running anything so you know which package starts.

06

`package-lock.json`

The exact dependency tree npm installed.

- Pins every transitive package version so installs are reproducible.
- Generated, never hand-edited - commit it, do not edit it.
- Huge diff usually means a dependency was upgraded, not a bug.

Server - boot and wiring

How the API process starts and how its parts are connected. If you are tracing a request, begin at index.ts.

07 **`server/package.json`**

Dependencies and scripts for the API package.

- dev runs tsx watch (auto-restart on save); start runs compiled JavaScript.
- The postinstall hook patches one dependency that crashes older Node versions.
- Production should use build + start, not the dev watcher.

08 **`server/tsconfig.json`**

TypeScript rules for the server, with strict mode on.

- NodeNext module resolution means imports carry a .js extension even in .ts files.
- noUncheckedIndexedAccess forces you to handle 'array[i] may be missing'.
- Strict settings catch real bugs; do not relax them to silence one error.

09 **`server/tsconfig.build.json`**

The same rules, but allowed to emit JavaScript into dist/.

- Only src/ is compiled - tests and scripts stay out of the build.
- This is what 'npm run build' and production 'npm start' use.

10 **`server/.env.example`**

The template for your local .env file - names only, no real secrets.

- Copy it to .env and fill in values; .env is never committed.
- Holds the port, CORS origins, database URL and model settings.

11 **`server/src/index.ts`**

The entry point: build the real dependencies, then listen on a port.

- Creates the Postgres pool, repositories, AI adapters and the LangGraph checkpointer.
- Calls migrate() at boot so a fresh database becomes usable without manual steps.
- Nothing here contains business logic - it is assembly only.

12 **server/src/app.ts**

Builds the Express app from injected dependencies and mounts the routers.

- Dependencies are parameters, so tests can pass fakes instead of Postgres.
- Adds CORS, JSON parsing, the error handler and the 404 fallback.
- One central error handler turns thrown ApiErrors into clean JSON responses.

13 **server/src/config.ts**

Reads environment variables once and validates them.

- Bad values fall back to safe defaults instead of crashing at random later.
- Every other file imports config, so environment handling lives in one place.

Server - domain

The vocabulary of the system: what a workflow step is, what makes a template valid, and how errors are shaped.

14 **`server/src/domain/schemas.ts`**

Zod schemas that define every shape crossing the API boundary.

- STEP_TYPES is the single vocabulary shared by API, database constraints and the AI prompt.
- Parsing with Zod both validates and strips unknown fields before anything is stored.
- Changing a rule here changes what the whole system accepts - treat it as a contract.

15 **`server/src/domain/businessRules.ts`**

Rules that decide whether a workflow makes sense, beyond having the right shape.

- Structural checks ask 'is this a valid step?'; these ask 'would this workflow work?'.
- Returns a list of violations instead of throwing, so the UI can show them all at once.
- Applies to AI drafts too - a generated template cannot skip them.

16 **`server/src/domain/errors.ts`**

One error type carrying an HTTP status, a code and a safe message.

- Small helpers (notFound, conflict, unprocessable) keep messages consistent.
- Never put a stack trace or credential in a message - it goes to the browser.
- Unexpected errors become a generic 500; only ApiError details reach the client.

Server - database

The connection pool, the migration runner, and the SQL files that create the schema in order.

17 **`server/src/db/pool.ts`**

Creates and configures the PostgreSQL connection pool.

- A pool reuses connections instead of opening one per request.
- Limits (max, timeouts) stop a slow database from exhausting the process.
- Only this file imports the pg driver for connections - repositories reuse the pool.

18 **`server/src/db/migrate.ts`**

A tiny forward-only migration runner.

- Runs each .sql file once, inside a transaction, and records its name in schema_migrations.
- Re-running is safe: already-applied files are skipped.
- Migrations only move forward; to undo, write a new migration.

19 **`server/src/db/migrations/001_init.sql`**

First schema: operators, templates, template steps and reviews.

- Idempotent (IF NOT EXISTS) so it can run on a fresh or existing database.
- CHECK constraints mirror the Zod enums so the database rejects bad rows too.
- Seeds the three demo identities: admin, operator, auditor.

20 **`server/src/db/migrations/002_workflow_instances.sql`**

Adds runs (instances) and their per-step history.

- A workflow_instance is one execution of an approved template.
- Steps are snapshotted at start, so editing the template cannot change a running run.
- step_runs records what happened in each step, including who did it and when.

21 **`server/src/db/migrations/003_document_execution.sql`**

Case data and document output for runs.

- Stores the input a run was started with; document steps render from it.
- Adds the columns needed to point at a generated file.

22 **server/src/db/migrations/003_role_model.sql**

Introduces the admin / operator / auditor role model.

- Replaces the older create-vs-approve split with three clear roles.
- Existing approver rows are migrated to operator so nobody loses access.
- Records a reason in the header - migrations are documentation too.

23 **server/src/db/migrations/004_operator_invites.sql**

Email whitelisting and revocable membership.

- Admins add people by email; that address becomes their sign-in identity.
- Removing access sets revoked_at instead of deleting the row.
- Kept rows mean the audit trail can always name the real person.

24 **server/src/db/migrations/005_audit_events.sql**

The append-only audit table.

- Each row stores a SHA-256 hash of its payload plus the previous row's hash.
- Editing or deleting a row breaks verification from that point onwards.
- Append-only: there is no update or delete path anywhere in the code.

25 **server/src/db/migrations/006_intake_steps.sql**

Lets execution history record intake steps.

- Widens the step_type CHECK constraint to include 'intake'.
- That is the whole change - intake needed no new table, only permission to exist.
- A reminder that constraints must be updated when a new step type appears.

Server - identity and permissions

Who is calling, and what may they do. This is the server-side half of every role check in the UI.

26 **`server/src/auth/operators.ts`**

The role model: one matrix mapping admin, operator and auditor to permissions.

- permissionsFor(role) returns strings like template:approve or workflow:respond.
- The UI hides buttons using the same list, but the server re-checks every action.
- Revoked members cannot authenticate, yet their history stays intact.

27 **`server/src/auth/middleware.ts`**

Express middleware that identifies the caller and enforces a permission.

- Reads the x-operator-id header and loads a live member record.
- requirePermission('x') returns 403 before the route body ever runs.
- The header is authorisation plumbing for the demo - swap it for session/JWT in production.

Server - audit trail

Recording what happened, in a way that cannot be quietly rewritten. This is the 'auditable' part of CASE.

28 **`server/src/audit/auditLog.ts`**

Reads and appends audit events, maintaining a SHA-256 hash chain.

- Each event's hash covers its own payload plus the previous hash (prevHash).
- A Postgres advisory lock serialises writers, so the chain never forks.
- verify() recomputes the chain and reports the first broken link.
- Canonical JSON matters: the same object must always hash identically.

29 **`server/src/audit/record.ts`**

A small helper routes call after a successful action.

- Fills in actor id and role so routes do not repeat that plumbing.
- Fire-and-forget: a logging failure is reported to the console, never to the user.
- The action already happened - failing the response would be a lie.

Server - AI

Language-model work: drafting templates, writing document content, and reading uploaded case files. Every model output is treated as untrusted input.

30 **`server/src/ai/generator.ts`**

Turns a plain-language request into a draft template, or asks a clarifying question.

- The prompt explains the allowed step types and the canonical step order.
- The reply is one flat envelope: either needsClarification or name + steps.
- Output is returned as unknown and re-validated with Zod - the model cannot bypass rules.

31 **`server/src/ai/documentContent.ts`**

Generates the structured content of a document step (title, sections, table).

- Schema mirrors the document builder, so the model fills fields the renderer understands.
- File names are decided server-side, so the model cannot influence where files are written.
- Same rule as templates: validate before use.

32 **`server/src/ai/fileReader.ts`**

Reads an uploaded PDF or image: text layer first, OCR when needed, then a summary.

- PDFs are read through pdfjs; scanned PDFs fall back to OCR of their embedded page images.
- Images are normalised first (greyscale, upscale, contrast) because raw phone photos read as noise.
- The OCR worker gets an errorHandler - without one a bad file kills the whole process.
- The summary prompt forbids inventing names, dates or amounts.

33 **`server/scripts/patch-wasm-feature-detect.mjs`**

A postinstall patch for one dependency that cannot parse on Node 22.

- The package ships import.source(), syntax only newer Node understands.
- The probe is unused by the OCR library, so it is replaced with a harmless false.
- Runs automatically after npm install and does nothing once the file is fine.

Server - workflow execution

LangGraph runs the steps of an approved template, pausing for human decisions and file uploads.

34 **`server/src/execution/types.ts`**

Types for running workflows: instance status, step status, gates, step runs.

- Defines PAUSED, COMPLETED, REJECTED and friends - the states the UI shows.
- Describes a pending gate: either an approval or an intake upload.
- Shared vocabulary keeps graph, service and repository in agreement.

35 **`server/src/execution/graph.ts`**

The LangGraph state machine that executes steps in order.

- approval and intake steps call interrupt(), which pauses the run until resumed.
- The graph is deterministic - no model decides the order of steps.
- Each node writes a step event so history can be rebuilt later.

36 **`server/src/execution/intake.ts`**

The port (interface) for reading uploaded files, plus a refusing default.

- Defines ReadFileInput and ReadFileResult so execution does not depend on tesseract or Ollama.
- UnavailableFileReader fails clearly when no reader is configured, e.g. in tests.
- Ports like this are why the AI can be swapped without touching the graph.

37 **`server/src/execution/documents.ts`**

Builds and stores the file a document step produces.

- Asks the content generator for structured content, then renders it through an exporter.
- File paths are derived from instance and step ids - never from model output.
- Returns the stored path and size so the run detail can offer a download.

38 **`server/src/execution/exporters.ts`**

Renders document content as a real PDF (pdfkit) or Word file (docx).

- Two exporters share one content shape, so a step can choose its format.
- Kept close to the CASE branch's builders so output looks the same.
- Pure functions of content -> bytes, which is why they are easy to test.

39 **server/src/execution/notifier.ts**

The outbound notification port and a simulation-only default.

- The default implementation just logs; nothing is ever really sent.
- Swap in an email or Telegram adapter to make notification steps real.
- A workflow containing a notification step must not message people by accident.

Server - repositories

The only layer that speaks SQL. Services above it work with plain objects.

40 **`server/src/repo/types.ts`**

Interfaces describing template storage: Template, Review, filters, updates.

- Services depend on this interface, not on Postgres.
- That is what allows an in-memory repository in tests.
- UUIDs and timestamps are strings here - the API is JSON, not Date objects.

41 **`server/src/repo/pgRepo.ts`**

PostgreSQL implementation for templates and their reviews.

- Parameterised SQL ($1, $2) - never string concatenation, which invites injection.
- Row objects are mapped to domain objects so snake_case never leaks upward.
- Uses a transaction and row lock where two writers could race.

42 **`server/src/repo/pgExecutionRepo.ts`**

PostgreSQL implementation for workflow instances and step runs.

- Stores the step snapshot taken when a run starts.
- Updates instance status and step rows as the graph progresses.
- Filtering (status, template) happens in SQL, not in memory.

Server - services

Business logic that does not belong in HTTP handlers: validation, state transitions, permissions of state.

43 **`server/src/services/templateService.ts`**

The template lifecycle: draft, edit, approve, reject, activate, deactivate.

- Runs schema validation, then business rules, before any write.
- Handles the AI path, including the clarification answer instead of a template.
- Enforces lifecycle rules such as 'only a PENDING template can be approved'.

44 **`server/src/services/instanceService.ts`**

Starts runs, resumes them, and turns gates into readable detail.

- Snapshots the template's steps so a later edit cannot change a running run.
- submitApproval and submitIntake resume the paused graph with the user's answer.
- Rejects actions that do not fit the current state, with a clear 409.

Server - routes

Thin HTTP layer: parse, authorise, call a service, return JSON. No business rules live here.

45 **`server/src/routes/http.ts`**

Small shared helpers for parsing params and wrapping async handlers.

- parse() turns Zod failures into a consistent 422 with field details.
- UuidParamSchema makes a malformed id a 404 instead of a database error.
- wrap() forwards async errors to Express's error handler.

46 **`server/src/routes/templates.ts`**

Endpoints under /api/templates: list, read, create, generate, update, approve, reject, deactivate.

- Each route names the permission it needs; that is the real access control.
- Audit events are recorded after the service succeeds.
- Validation is delegated - handlers stay readable.

47 **`server/src/routes/instances.ts`**

Endpoints under /api/instances: start, read, respond, cancel and the intake upload.

- The upload route uses multer in memory, with a 10 MB limit.
- A 413/400 is returned for oversized or unreadable uploads, never a crash.
- Resuming only happens after the file was actually read.

48 **`server/src/routes/operators.ts`**

Admin endpoints for membership: invite, change role, revoke, and /me for the caller.

- /me returns the member plus their permission list, which the UI mirrors.
- Inviting an existing revoked email restores access instead of duplicating rows.
- Membership changes are audited like any other privileged action.

49 **`server/src/routes/audit.ts`**

Read-only endpoints for the audit trail and its verification.

- Supports paging and filters (action, outcome, actor).
- verify recomputes the hash chain and reports the first broken index.
- There is deliberately no write endpoint - events are appended elsewhere.

Server - tests

Vitest suites that exercise the API over real HTTP, with in-memory repositories standing in for Postgres.

50 **`server/tests/validation.test.ts`**

Pure tests for schemas and business rules.

- No server, no database - just inputs and expected violations.
- Fastest suite, so it runs first when you change a rule.

51 **`server/tests/lifecycle.test.ts`**

End-to-end template lifecycle over HTTP.

- Boots the Express app on a random port once, then makes real requests.
- Covers create, edit, approve, reject and the AI clarification path.

52 **`server/tests/execution.test.ts`**

Runs a workflow to completion, including pausing and resuming.

- Uses a fake generator and the real graph.
- Asserts on step history and on the generated document file.

53 **`server/tests/documents.test.ts`**

Checks document content validation and the file builder.

- Writes into a temporary directory that is deleted afterwards.
- Proves the model's content cannot escape its schema.

54 **`server/tests/roles.test.ts`**

The permission matrix, tested from the outside.

- Auditor is refused mutations, operator is allowed, admin manages members.
- This suite is the contract the UI is expected to mirror.

55 **`server/tests/support/memoryRepo.ts`**

In-memory template repository for tests.

- Implements the same interface as the Postgres version.
- Lets tests run in milliseconds without a database.

56 **server/tests/support/memoryExecutionRepo.ts**

In-memory store for instances and step runs.

- Same interface as the SQL repository, backed by Maps.

57 **server/tests/support/operators.ts**

In-memory operator directory with the admin write operations.

- Seeds members so a test can log in as any role.

Web console

The React app in web/: a compact console for templates, runs, members and the audit trail. Vite builds it.

58 **`web/index.html`**

The single HTML page the browser loads.

- Contains one empty div; React fills it.

59 **`web/vite.config.ts`**

Vite + React + Tailwind configuration.

- Dev server on port 5173; no proxy, the API URL is set in src/api.ts.

60 **`web/package.json`**

Web dependencies and scripts.

- dev starts Vite, build runs a typecheck then bundles.

61 **`web/tsconfig.json`**

TypeScript settings for the web package.

- Strict mode plus JSX settings; build runs tsc before Vite bundles.

62 **`web/src/main.tsx`**

Mounts React into the page.

- Three lines of bootstrap; the app itself lives in App.tsx.

63 **`web/src/index.css`**

Tailwind entry plus base page styles.

- @import 'tailwindcss' is all Tailwind needs.

64 **`web/src/api.ts`**

Every HTTP call the UI makes, in one place.

- Sends x-operator-id so the server knows who is acting.
- Turns error responses into a typed ApiError for the components.
- One file means a changed endpoint is fixed once.

65 **web/src/types.ts**

Shared TypeScript shapes mirrored from the server.

- If a server field changes, update this file - the compiler will point at every use.

66 **web/src/App.tsx**

App shell: identity switcher, navigation and data loading.

- Loads /api/operators/me to learn the permission list.
- Passes canEdit / canApprove / canRespond down to hide buttons.
- Hiding a button is convenience; the server still enforces the rule.

67 **web/src/components/ui.tsx**

Small shared presentational pieces: badges and error boxes.

- Pure display components with no state or fetching.

68 **web/src/components/TemplateList.tsx**

The template table with filters and status badges.

- Receives data and callbacks; it never fetches on its own.

69 **web/src/components/TemplateDetail.tsx**

One template: steps, review status, and the approve/reject actions.

- Shows edit controls only when the role allows it.
- Renders server-side validation messages from a 422 response.

70 **web/src/components/StepsEditor.tsx**

Add, remove and reorder workflow steps.

- Produces the steps array the API expects.
- Validation still happens on the server - this only shapes the input.

71 **web/src/components/CreatePanels.tsx**

Two ways to start a template: build by hand, or describe it to the AI.

- Handles the clarification answer: the AI may reply with a question instead of a template.

72 **web/src/components/RunsView.tsx**

List of runs with a start action.

- Only approved, active templates can be started.

73 **web/src/components/RunDetail.tsx**

One run: the step timeline, approval form, intake upload and generated document.

- Renders the paused gate the server reports - approval form or file upload.
- Uploads go through multipart; a successful upload resumes the run.
- Documents are fetched with the identity header, then shown from a blob URL.

74 **web/src/components/Members.tsx**

Admin-only membership management.

- Invite by email, switch role, revoke access.
- Revoked members stay visible so history keeps its owner.

Appendix - the dashboard app

The larger CASE console lives outside this repository, in ../dashboard. These are its files, one line each.

75 **dashboard/src/main.tsx**

Browser bootstrap: mounts React.

76 **dashboard/src/App.tsx**

Routes between Dashboard, Workflows, Runs, Employees, Audit, Integrations and Settings.

77 **dashboard/src/api/client.ts**

Fetch wrapper: identity header, error mapping, one function per endpoint.

78 **dashboard/src/api/types.ts**

Server shapes mirrored for the dashboard.

79 **dashboard/src/context/SessionContext.tsx**

Who is signed in: stores identity, loads /me, exposes can(permission).

80 **dashboard/src/features/auth/Login.tsx**

Sign-in screen: whitelist check against the server.

81 **dashboard/src/features/dashboard/DashboardView.tsx**

Overview: counts, active workflows and the team table.

82 **dashboard/src/features/workflows/WorkflowsView.tsx**

Workflow list, AI generation panel and role-gated actions.

83 **dashboard/src/features/workflows/WorkflowEditor.tsx**

Node canvas: draggable step cards, connectors, positions saved with the template.

84 **dashboard/src/features/workflows/useWorkflowData.ts**

Data hook: loads templates/instances and runs mutations.

85 **dashboard/src/features/runs/RunDetail.tsx**

One run: timeline, approval gate and intake upload.

86 **dashboard/src/features/audit/AuditView.tsx**

Audit trail browser with hash-chain verification.

87 **dashboard/src/features/employees/EmployeeView.tsx**

Members and roles, admin only.

88 **dashboard/src/features/settings/SettingsView.tsx**

Local model connection test and workspace settings.